

Licenciatura en Sistemas

Trabajo Práctico HARRY POTTER - API

INTRODUCCIÓN A LA PROGRAMACIÓN

(comisión verano 2025)

Resumen

El trabajo práctico consiste en la implementación de una aplicación web que permite a los usuarios explorar y visualizar imágenes de personajes obtenidas de una API de Harry Potter. La aplicación no solo proporciona una interfaz intuitiva para navegar por los distintos personajes, sino que también permite a los usuarios autenticados marcar y gestionar sus imágenes favoritas, ofreciendo así una experiencia personalizada.

La aplicación está desarrollada utilizando Python y el framework Django, siguiendo una arquitectura bien estructurada en diferentes capas. Esta organización tiene como objetivo mantener una clara separación de responsabilidades, lo que facilita el mantenimiento, la escalabilidad y la reutilización del código. Entre las capas principales de la aplicación se incluyen la de presentación (interfaz de usuario), la de lógica de negocio y la de acceso a datos, asegurando un desarrollo modular y eficiente.

Integrantes

- **Rodríguez Cristian**, crisrodriguez.dev@gmail.com
- **Vargas Jorge**, Jorgevargas20202005@gmail.com

Introducción

El trabajo práctico consiste en la implementación de una aplicación web que muestra imágenes de la API de Harry Potter y permite a los usuarios autenticados marcar y gestionar sus imágenes favoritas. La aplicación está desarrollada utilizando Python, Django y sigue una arquitectura que separa claramente las responsabilidades entre diferentes capas: presentación, servicio, transporte y acceso a datos.

A continuación, se presenta el código de las funciones implementadas, una breve explicación de cada una de ellas, las dificultades encontradas durante la implementación y las decisiones tomadas para resolverlas.

Desarrollo

Implementación de los códigos obligatorios:

- views.py:

```
# esta función obtiene 2 listados: uno de las imágenes de la API y otro de
# favoritos, ambos en formato Card, y los dibuja en el template 'home.html'.

def getAllImagesAndFavouriteList(request):
    #obtenemos todas las imagenes de la API
    images = services.getAllImages()

    if request.user.is_authenticated:
        favourite_list = services.getAllFavourites(request.user)
    else:
        favourite_list = []

    return images, favourite_list

def home(request):
    # Llama a la funcion auxiliar getAllImagesFavouriteList() y obtiene 2
    # listados: uno de las imágenes de la API y otro de favoritos por usuario
    images, favourite_list = getAllImagesAndFavouriteList(request)

    return render(request, 'home.html', { 'images': images, 'favourite_list':
favourite_list })
```

- services.py:

```
# función que devuelve un listado de cards. Cada card representa una imagen de
la API de Harry Potter.
def getAllImages():

    json_collection = transport.getAllImages()
    images = []

    for object in json_collection:
        card = translator.fromRequestIntoCard(object)

        # Seleccionar un nombre alternativo al azar, si existen.
        if "alternate_names" in object and object["alternate_names"]:
            card.alternate_name = random.choice(object["alternate_names"])
        else:
            card.alternate_name = "No tiene apodos"

        images.append(card)

    return images
```

- home.html:

```
<div class="row row-cols-1 row-cols-md-3 g-4">
    {% if images|length == 0 %}
    <h2 class="text-center">La búsqueda no arrojó resultados...</h2>
    {% else %} {% for img in images %}
    <div class="col">
    <!-- evaluar si la imagen pertenece a Gryffindor, Slytherin u otro -->
        <div class="card border border-3 {% if img.house == 'Gryffindor' %}border-success{% elif img.house == 'Slytherin' %}border-danger{% else %}border-warning{% endif %} mb-3 ms-5" style="max-width: 540px; ">

            <div class="row g-0">
                <div class="col-md-4">
                    
                </div>
```

Explicación de las Funciones:

- *def home (views.py)*

La función home llama a getAllImagesAndFavouriteList, que obtiene dos listados: uno con las imágenes disponibles por la API de Harry Potter, y otro con las imágenes favoritas del usuario (siempre que el usuario esté registrado; de lo contrario, retorna una lista vacía). Luego, pasa estas listas al template home.html para que se puedan visualizar.

- *def getAllImages (services.py)*

Retorna todas las imágenes de la API de Harry Potter. Para eso, se usa services.getAllImages(), que obtiene los datos de la API, los convierte en objetos Card y los almacena en un listado para su uso posterior.

- *home.html (template)*

La línea a la cual le damos énfasis en esta página para lograr el cambio de color dependiendo de la familia, es:

```
<div class="card {% if img.house == 'Gryffindor' %}border-success{% elif img.house == 'Slytherin' %}border-danger{% else %}border-warning{% endif %} mb-3 ms-5" style="max-width: 540px; ">
```

Se usa un contenedor <div> en HTML, indicamos una clase llamada “card” (representa una tarjeta con estilos de Bootstrap), después entre los códigos {%....%}: Es una plantilla de Django que usa Python para decidir que clases de CSS agregar. También adentro de los códigos, utilizamos condicionales como if, elif y else, que dependiendo de si su casa sea Slytherin va hacer color rojo, o si su casa es Gryffindor será de color verde y si no es ninguna de las dos, será color amarillo.

Implementación de los códigos opcionales:

-views.py:

Buscador I (según el nombre del personaje) ☆

```
# función utilizada en el buscador.
def search(request):

    name = request.GET.get('query', '')
    images, favourite_list = getAllImagesAndFavouriteList(request)

    # si el usuario ingresó algo en el buscador, se deben filtrar las imágenes
    por dicho ingreso.
```

```
images_coin = []

if name:
    for img in images:
        if name.lower() in img.name.lower():
            images_coin.append(img) #Agregamos la img a la lista images=[]

    favourite_list = []
    return render(request, 'home.html', { 'images': images_coin,
'favourite_list': favourite_list })
else:
    return redirect('home')
```

Buscador II (filtro según casa) ☆

```
# función utilizada para filtrar por casa Gryffindor o Slytherin.
def filter_by_house(request):

    house = request.POST.get('house', '')
    images, favourite_list = getAllImagesAndFavouriteList(request)

    images_coin = [] # debe traer un listado filtrado de imágenes, según la
casa.

    if house:
        for img in images:
            if house == img.house:
                images_coin.append(img)
        favourite_list = []

        return render(request, 'home.html', { 'images': images_coin,
'favourite_list': favourite_list })
    else:
        return redirect('home')
```

- Buscador por nombre(función **search**)

Si el usuario escribe un nombre, se deben mostrar solo las imágenes que coincidan con ese nombre. Si el usuario no escribe nada, se deben mostrar todas las imágenes, como si hiciera clic en "Galería".

name = request.GET.get('query', '') , busca en la URL lo que el usuario escribió en el campo del buscador, si el usuario no escribió nada, se asigna un texto vacío ''.

Se obtienen todas las imágenes de la API. Se crea una lista vacía para guardar las imágenes filtradas, y recorremos todas las imágenes a través de un for. Si el texto que el usuario escribió está dentro de una imagen `img.name` esa imagen se agrega a `images_coin`.

Por último se envían los resultados a `template home.html` a través del `return` (enviamos la lista de `images_coin`) y si el usuario no escribió nada en el buscador retornamos a la página "galería" (home), y se muestran todas las imágenes.

- *Buscador II (filtro según casa)*

Cuando el usuario hace clic en una casa específica (por ejemplo, "Gryffindor" o "Slytherin"), el sistema filtra las imágenes de los personajes que pertenecen a esa casa.

Para lograrlo, primero se obtiene la casa seleccionada a través del formulario. Esto se hace con `request.POST.get('house', '')`, que recupera el valor de la casa elegida. Si no se seleccionó ninguna casa, el valor devuelto será un texto vacío.

Luego, se obtienen todas las imágenes de la API junto con la lista de imágenes favoritas del usuario llamando a la función `getAllImagesAndFavouriteList(request)`.

Después, se crea una lista vacía llamada `images_coin`, que almacenará las imágenes filtradas.

Si el usuario seleccionó una casa, se recorre la lista de imágenes disponibles y se verifica si la casa de cada imagen coincide con la casa elegida por el usuario. Si hay coincidencia, la imagen se agrega a `images_coin`.

Una vez que se han filtrado las imágenes, se envían al archivo `home.html`, donde se mostrarán solo las que corresponden a la casa seleccionada.

En caso de que el usuario no haya seleccionado ninguna casa, se redirige automáticamente a la página principal (home), mostrando todas las imágenes disponibles sin ningún filtro aplicado.

- `home.html`:

Loading Spinner - para la carga de imágenes

```
<!-- Pantalla de carga -->
<div id="loading-screen" style="position: fixed; top: 0; left: 0; width:
100%; height: 100%; background: rgba(255, 255, 255, 0.9); display: none;
justify-content: center; align-items: center; z-index: 9999;">
  <div class="spinner-border text-primary" role="status">
    <span class="visually-hidden">Cargando...</span>
```

```
        </div>
    </div>

#Al final de la hoja home.html
<script>
    document.addEventListener("DOMContentLoaded", function () {
        // Captura los formularios de búsqueda y filtrado
        let searchForm = document.querySelector(".search-form");
        let filterForms = document.querySelectorAll(".filter-form");
        let loadingScreen = document.getElementById("loading-screen");

        // Muestra el loader cuando se envía el formulario de búsqueda
        searchForm.addEventListener("submit", function () {
            loadingScreen.style.display = "flex";
        });

        // Muestra el loader cuando se envía un formulario de filtro
        filterForms.forEach(function (form) {
            form.addEventListener("submit", function () {
                loadingScreen.style.display = "flex";
            });
        });

        // Oculta el loader cuando la página ha cargado
        window.addEventListener("load", function () {
            loadingScreen.style.display = "none";
        });
    });
</script>
```

- *Loading Spinner (para la carga de imágenes)*

Cuando un usuario busca un personaje o filtra por casa en la página, es posible que las imágenes tarden un momento en cargarse. Para mejorar la experiencia del usuario, se implementó una pantalla de carga (loading-screen), que es un fondo semitransparente con un círculo giratorio (spinner) en el centro. Esta pantalla se mantiene oculta al inicio, pero se muestra automáticamente cuando el usuario realiza una acción que requiere cargar nuevas imágenes, como buscar un personaje o seleccionar una casa específica.

El script asociado a esta funcionalidad se activa en dos momentos clave. Primero, cuando el usuario presiona el botón de búsqueda o alguno de los botones de filtrado, el div con el id="loading-screen" cambia su propiedad display de none a flex, haciéndolo visible y mostrando el spinner. De esta forma, el usuario sabe que la página está procesando su solicitud.

Luego, una vez que la página ha terminado de cargar todas las imágenes, el evento `window.load` se ejecuta automáticamente y vuelve a ocultar la pantalla de carga, permitiendo al usuario ver los resultados sin interrupciones. Gracias a esta implementación, se evita que el usuario piense que la página está inactiva, ya que recibe una señal visual clara de que el contenido está en proceso de carga.

Además, de los puntos obligatorios y opcionales, agregamos y mejoramos la página web de Harry Potter con algunos puntos tales como:

- 1) Imágenes de fondo en los templates de `home.html`, `index.html` y `login.html`.
- 2) Bordes más gruesos en las tarjetas de los personajes así logrando una mejor distinción de sus casas.
- 3) Cambio de color en el navbar.
- 4) Cambio de ubicación y mejora de diseño del login.

```
1) <body style="background-image: url('{% static 'img/collage.jpg' %}');  
    background-size:cover background-position: center;">
```

En este punto establecemos una imagen de fondo para la página.

Para que funcione correctamente, primero debemos guardar nuestras imágenes en la carpeta `static/img/`. Dentro de nuestro proyecto de Django, ubicamos la carpeta `static` y dentro de ella creamos otra llamada `img`. Luego, colocamos dentro la imagen que queremos usar, en este caso, `collage.jpg`.

Para poder acceder a los archivos dentro de la carpeta `static`, es necesario cargar los archivos estáticos en la plantilla utilizando `{% load static %}` al inicio del archivo HTML.

Después, en la etiqueta `<body>`, usamos `{% static 'img/collage.jpg' %}` para obtener la URL correcta de la imagen y aplicarla como fondo. Además, agregamos algunas reglas de CSS.

```
2) <div class="card border border-3 {% if img.house == 'Gryffindor'  
    %}border-success{% elif img.house == 'Slytherin' %}border-danger{% else  
    %}border-warning{% endif %} mb-3 ms-5" style="max-width: 540px; ">
```

En este punto agregamos estilo CSS directamente desde el `div`, el estilo que cumple esta función es `border border-3` que le da un grosor mas grueso para que pueda resaltar el marco de la tarjeta “card”.

```
3) <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
```


Aquí aplicamos la misma regla que el punto anterior, trabajamos con clases de Bootstrap5 dentro de <nav>, usamos navbar que define el elemento como barra de navegación, navbar-expand-lg que permite que la barra sea responsive y se expanda en pantallas grandes, navbar-dark (el estilo que agregamos principalmente) que cambia el color del texto y los iconos a un tono mas claro y por último bg-dark, aplica un fondo oscuro a la barra de navegación.

```
4) <div class="login-form d-flex justify-content-center align-items-
    center" style="text-align: center; margin-top: 25vh;">
    <form style="background-color: rgb(252, 252, 252); padding: 25px;
border-radius: 10%;" action="" method="POST" style="display: inline-block;">
        {% csrf_token %}
        <h2 class="text-center; padding:5px;">Inicio de sesión</h2>
        <div class="form-group" style="margin-bottom: 5%;">
            <input type="text" name="username" id="username" class="form-
control" placeholder="Usuario" required="required">
        </div>
        <div class="form-group" style="margin-bottom: 5%;">
            <input type="password" name="password" id="password"
class="form-control" placeholder="Contraseña" required="required">
        </div>
        <div class="form-group">
            <button type="submit" class="btn btn-primary btn-
block">Ingresar</button>
        </div>
    </form>
</div>
```

En este código, rediseñamos el formulario de inicio de sesión usando Bootstrap y estilos CSS en línea para centrarlo y mejorar su apariencia.

El div con class="login-form" usa Flexbox para centrar el formulario en la pantalla y en el form usamos un fondo blanco con bordes redondeados y padding para una mejor apariencia.

Dificultades de Implementación y Decisiones Tomadas:

Inicialmente, enfrentamos dificultades al instalar los programas necesarios para ejecutar el trabajo en nuestros equipos. Una vez superada esta fase, seguimos las

instrucciones del profesor para visualizar la página en el navegador y analizamos su funcionamiento.

Después, revisamos el código completo usando Visual Studio Code, intentando comprender su lógica general antes de completar las secciones faltantes. Identificamos que las funciones debían llamarse entre sí en un flujo lógico:

1. La función en `views.py` recibe los datos desde `services.py`.
2. `services.py` obtiene los datos desde `transport.py` y los convierte en Cards.

Otra dificultad fue integrar correctamente las funciones entre las distintas capas de la arquitectura. Optamos por dividir la obtención de datos en funciones auxiliares, lo que facilitó la depuración y el mantenimiento del código.

Finalmente, resolvimos la transformación de objetos en Cards y su almacenamiento en un listado de imágenes. La decisión de centralizar esta lógica en el módulo `translator.py` permitió mantener el código organizado y reutilizable.

Conclusión: el proyecto implicó una serie de decisiones técnicas y desafíos que fueron abordados mediante una clara separación de responsabilidades y una estructura del código bien definida. Esto facilitó la implementación, prueba y mantenimiento de la aplicación.

Anexo

Implementar las funciones restantes de los archivos **views.py**, **services.py** y modificar **home.html**. Éstas son las encargadas de hacer que las imágenes de la galería se muestren/rendericen:

- **views.py**:

- *home(request)*: obtiene 2 listados que corresponden a las imágenes de la API y los favoritos del usuario, y los usa para dibujar el correspondiente template. Si el opcional de favoritos no está desarrollado, devuelve un listado vacío.

```
# esta función obtiene 2 listados: uno de las imágenes de la API y otro de
# favoritos, ambos en formato Card, y los dibuja en el template 'home.html'.
def home(request):
    images = []
    favourite_list = []

    return render(request, 'home.html', { 'images': images, 'favourite_list':
favourite_list })
```

- **Services.py**:

- *getAllImages*: obtiene un listado de imágenes convertidas en cards desde la API.

```
# función que devuelve un listado de cards. Cada card representa una imagen de
# la API de HP.
def getAllImages():
    # debe ejecutar los siguientes pasos:
    # 1) traer un listado de imágenes crudas desde la API (ver transport.py)
    # 2) convertir cada img. en una card.
    # 3) añadirlas a un nuevo listado que, finalmente, se retornará con todas
    # las card encontradas.
    # ATENCIÓN: contemplar que los nombres alternativos, para cada personaje,
    # deben elegirse al azar. Si no existen nombres alternativos, debe mostrar un
    # mensaje adecuado.
    pass
```

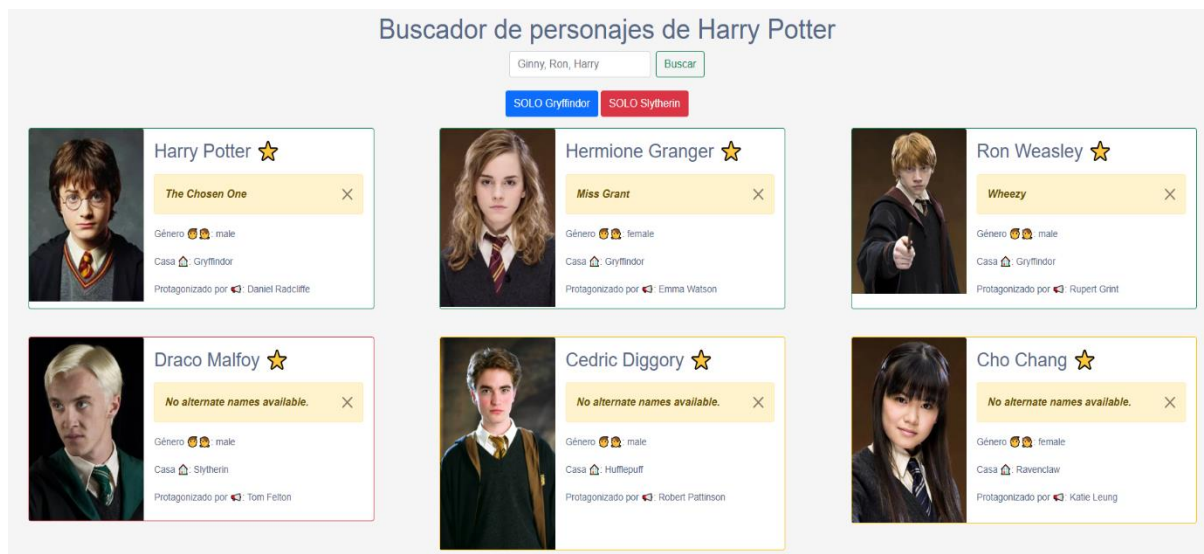
- **Home.html**:

- La card debe cambiar su border color dependiendo de la casa del personaje. Si pertenece a **Gryffindor**, mostrará un borde verde; si es de **Slytherin** mostrará rojo y si es de cualquier otra casa, será naranja. Se

sugiere consultar la **documentación de Bootstrap sobre Cards** y **cómo generar condicionales en Django** para tener un mejor acercamiento a la solución.

```
<div class="row row-cols-1 row-cols-md-3 g-4">
  {% if images|length == 0 %}
    <h2 class="text-center">La búsqueda no arrojó resultados...</h2>
  {% else %} {% for img in images %}
    <div class="col">
      <!-- evaluar si la imagen pertenece a Gryffindor, Slytherin u otro -->
      <div class="card border-success mb-3 ms-5" style="max-width:
540px;">
        <div class="row g-0">
          <div class="col-md-4">
            
          </div>
```

Concluido su desarrollo, deberían ver algo como lo siguiente:



Opcionales:

Las siguientes funcionalidades del juego NO son necesarias para la aprobación (con nota mínima 4), pero sirven para mejorar la calificación. De optar por hacerlas, se aplican las mismas reglas y criterios de corrección respecto de las funcionalidades básicas.

- **Buscador I (según el nombre del personaje) ☆**

Se debe completar la funcionalidad para que el buscador filtre adecuadamente las imágenes, según los siguientes criterios:

Si el usuario NO ingresa dato alguno y hace clic sobre el botón 'Buscar', debe mostrar las mismas imágenes que si hubiese hecho clic sobre el enlace Galería.

Si el usuario ingresa algún dato (ej. Harry), al hacer clic en 'Buscar' se deben desplegar las imágenes filtradas relacionadas a dicho valor.

Ejemplo para Weasley. ATENCIÓN, las imágenes pueden variar:



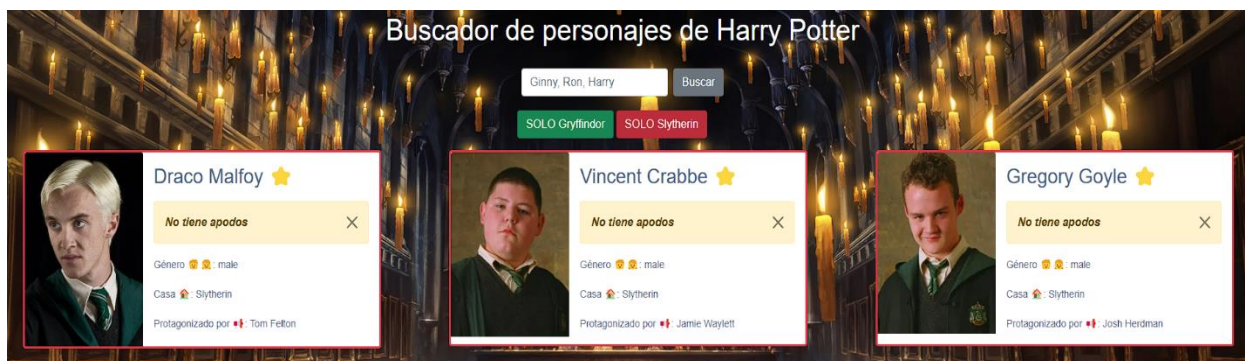
- **Buscador II (filtro según casa) ☆**

Se debe completar la funcionalidad para que los 2 botones de filtrado de resultados funcionen, según los siguientes criterios:

Si se hace clic sobre SOLO Gryffindor, debe mostrar resultados de personajes con dicha casa de estudio.

Ídem para el caso de SOLO Slytherin.

Ejemplo para Slytherin. ATENCIÓN, las imágenes pueden variar:



- **Loading Spinner para la carga de imágenes**

Se desea implementar una pantalla de carga/loading spinner que indique el usuario que espere hasta que la carga de imágenes se complete.

Una posible visualización de este ítem resuelto es:

